Pointers

Introduction to Pointers....1
Size of Pointers....9
Why Pointers are Important in Embedded Systems....13
Pointer Arithmetic....15
Types of Pointers....17
1)Null Pointer....17
2)Wild Pointer....23
3)Dangling Pointer....27
4)Void Pointer....34
5)Pointer to Pointer....41
6)Array and Pointer Relationship....42
7)Array Pointer....53
8)Array of Pointers....54
9)Function Pointer....54
10)Constant Pointer....55
11)Pointer to Constant....56
12)Constant Pointer to Constant....57
13)Structure Pointer....57
14)Returning Pointer from Function....58
Memory Mapped I/O Using Pointer....59
volatile with Pointer....59
Pointer to Volatile....59
Tricky Pointer Expressions....60
Near, Far and Huge Pointer....61
Practice Questions....61
Basic Questions....61
Advanced Questions....61
Practice Programs....62

Introduction to Pointers

A pointer is one of the most important concepts in C programming. A pointer is a variable that stores the address of another variable.

Normally, variables store data.

Example: int x = 10;

Here, x stores the value 10

But a pointer stores an address.

Example: int *ptr = &x;

Here a ptr stores address of x

Simple Real Life Example

Suppose:

- Your house is a variable
- House number is address
- Pointer stores house number

Pointer does not store actual house. It stores location of house.

Why Pointers are Important

Pointers are heavily used in:

- Embedded Systems
- Device Drivers
- Operating Systems
- RTOS
- Communication Protocols
- Dynamic Memory Allocation
- DMA
- Interrupt Handling
- Hardware Register Access

Without pointers, embedded programming is almost impossible.

Memory Basics

Every variable is stored somewhere in memory.

Each memory location has:

- Address

- Value

Example:

int x = 10;

Suppose memory looks like:

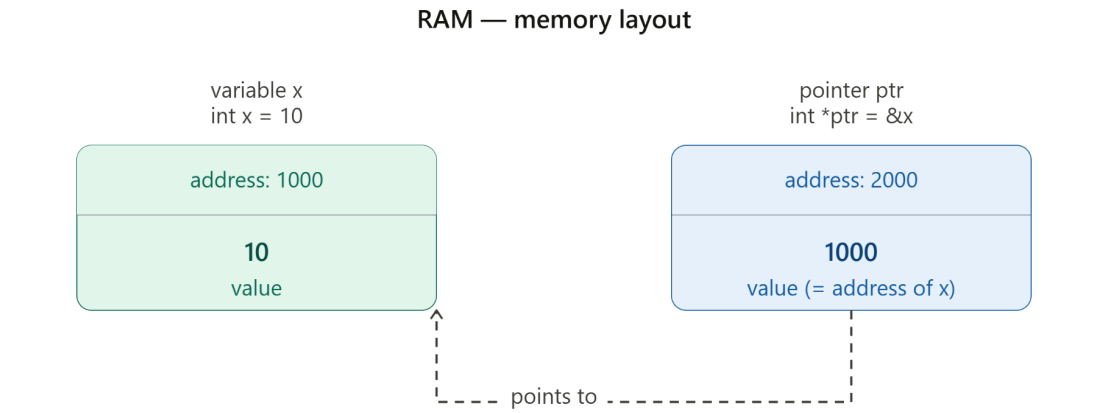
Address	Value
1000	10

Now:

int *ptr = &x;

Then:

Variable	Stored Value
ptr	1000



How it works

1. int x = 10 → allocates space at address 1000 and stores value 10
2. &x → the address-of operator returns 1000
3. int *ptr = &x → ptr is allocated at address 2000 and stores 1000

Pointer Declaration

Syntax

data_type *pointer_name;

Examples

int *p;
char *c;
float *f;
double *d;

NOTE: The data type of the pointer should match the data type of the variable.

Correct:

int x = 10;
int *p = &x;

Incorrect: float *p = &x;

Address Operator (&)

Address operator returns memory address of variable.

Example: int x = 10;

printf("%p", &x);

Output may be: 1000

Dereference Operator (*)

The **dereference operator (*)** is used to **access the value stored at the memory address contained in a pointer**.

Syntax

*pointer_name

Here, pointer_name contains an address, and *pointer_name gives the value stored at that address.

Example 1

```c
#include <stdio.h>

int main()
{
    int a = 10;      // Declare an integer variable and initialize it to 10
    int *p;          // Declare a pointer variable that can store the address of an integer

    p = &a;          // Store the address of variable 'a' in pointer 'p'

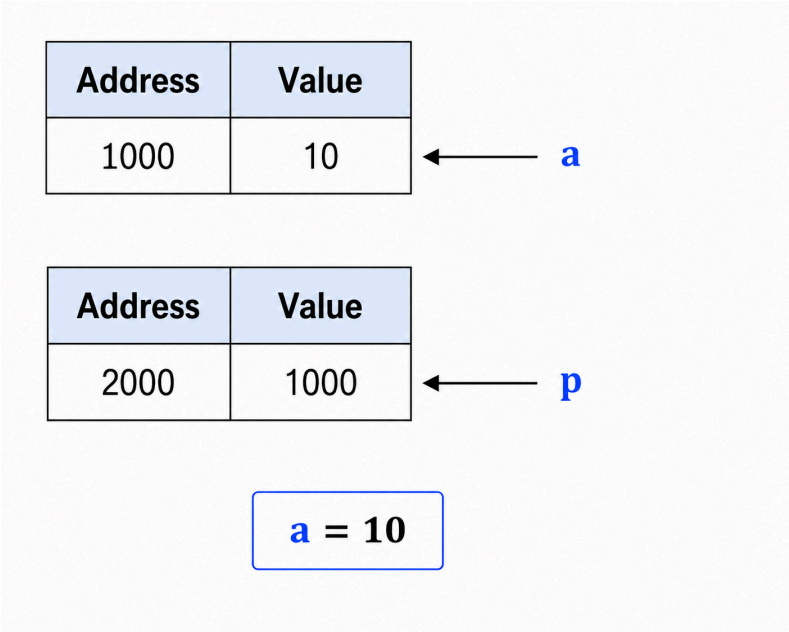
    printf("Value of a = %d\n", a);                 // Print the value of a
    printf("Address of a = %p\n", (void *)p);       // Print the address stored in p
    printf("Value at address stored in p = %d\n", *p); // Dereference p to print the value of a

    return 0;        // Indicate successful program execution
}
```

Memory Diagram

Address Value
1000 10 ← a

Address Value
2000 1000 ← p
a = 10



p = 1000 (address of a)

*p = 10 (value stored at address 1000)

Example 2: Modifying a Variable Using Dereference Operator

```c
#include <stdio.h>

int main()
{
    // Declare an integer variable and initialize it with 10
    int a = 10;

    // Declare a pointer and store the address of 'a'
    int *p = &a;
```

```c
    // Change the value of 'a' using the pointer
    *p = 50;

    // Display the updated value of 'a'
    printf("a = %d", a);

    return 0;
}
```

Output
a = 50

Explanation
*p = 50;

Means it store 50 at the address contained in p. Since p contains the address of a, the value of a becomes 50.

Step-by-Step

int a = 25;
int *p = &a;

Expression	Meaning
a	Value of variable (25)
&a	Address of variable
p	Address stored in pointer
*p	Value at the address stored in pointer

Suppose a is stored at address 1000

Then:

a = 25
&a = 1000
p = 1000
*p = 25

Important Difference

Address-of Operator (&)

Returns the address of a variable.

p = &a;
&a → Address of a

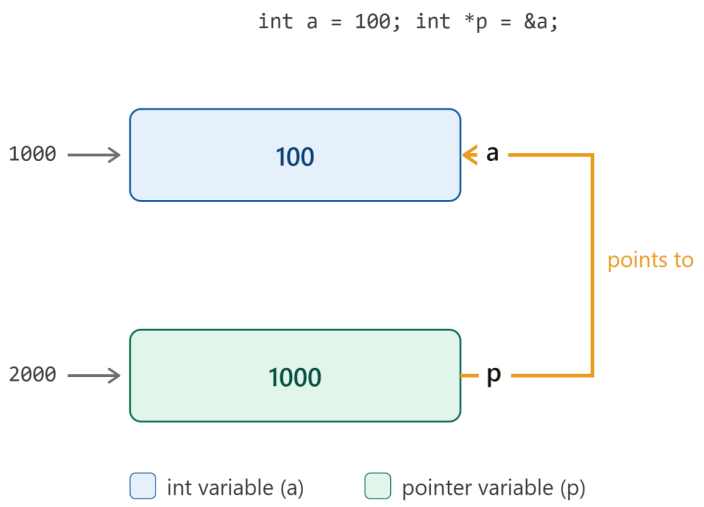
Dereference Operator (*)

Returns the value stored at that address.
*p → Value stored at address contained in p

Memory Representation

int a = 100;
int *p = &a;

p → pointer, a → variable



p = 1000 (1000 is a address of a)

*p = 100 (100 is a value stored at address 1000)

Complete Basic Pointer Program

```c
#include <stdio.h>

int main()
{
    // Declare an integer variable and initialize it with 10
    int x = 10;

    // Declare an integer pointer
    int *ptr;

    // Store the address of variable 'x' in pointer 'ptr'
    ptr = &x;

    // Print the value of variable 'x'
    printf("Value of x = %d\n", x);

    // Print the address of variable 'x'
    printf("Address of x = %p\n", (void *)&x);

    // Print the address stored inside pointer 'ptr'
    printf("Value stored inside ptr = %p\n", (void *)ptr);

    // Dereference 'ptr' to access the value stored at the address
    printf("Value pointed by ptr = %d\n", *ptr);

    return 0;
}
```

Program Explanation

int x = 10; → Creates integer variable.
int *ptr; → Creates integer pointer.
ptr = &x; → Stores address of x into ptr.
*ptr → Accesses value present at address stored inside ptr.

Size of Pointers

A **pointer variable stores the memory address of another variable**. Since it stores an address, the size of a pointer depends on the computer architecture (16-bit, 32-bit, or 64-bit), **not on the data type it points to**.

Syntax

data_type *pointer_name;

Examples:

int *p1;
char *p2;
float *p3;
double *p4;

All of these store memory addresses.

Program

```c
#include <stdio.h>

int main()
{
   int *p1;
   char *p2;
   float *p3;
   double *p4;

   printf("Size of int pointer    = %u bytes\n", sizeof(p1));
   printf("Size of char pointer   = %u bytes\n", sizeof(p2));
   printf("Size of float pointer  = %u bytes\n", sizeof(p3));
   printf("Size of double pointer = %u bytes\n", sizeof(p4));

   return 0;
}
```

Output on a 32-bit System

Size of int pointer   = 4 bytes
Size of char pointer   = 4 bytes
Size of float pointer = 4 bytes
Size of double pointer = 4 bytes

Output on a 64-bit System

Size of int pointer   = 8 bytes
Size of char pointer   = 8 bytes
Size of float pointer = 8 bytes
Size of double pointer = 8 bytes

Why Are All Pointer Sizes Equal?

Consider:

```
int *p1;
char *p2;
float *p3;
```

Although they point to different data types, they all store only an **address**.

Example:

```
p1 = 1000
p2 = 2000
p3 = 3000
```

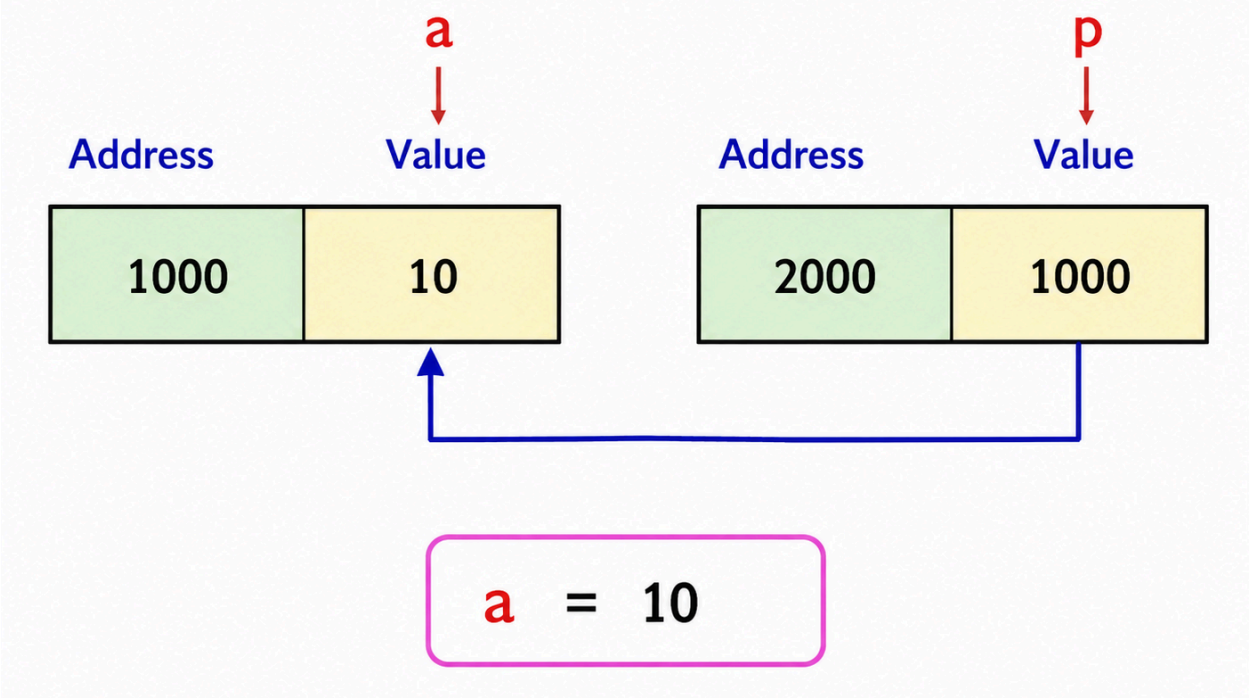
Each variable stores an address, so the amount of memory required is the same.

Memory Representation

```
int num = 10;
int *p = &num;
```

Suppose:

num is stored at address 1000. And the pointer is stored at address 2000.



Here p stores the address 1000. The size of p depends on how many bytes are required to store an address.

Pointer Size vs Data Size

int num;
int *p;

Item	Size (32-bit System)
int	4 bytes
int*	4 bytes

char ch;
char *p;

Item	Size (32-bit System)
char	1 byte
char*	4 bytes

double d;
double *p;

Item	Size (32-bit System)
double	8 bytes
double*	4 bytes

Notice that the size of the pointer remains the same even though the data sizes are different.

Architecture-wise Pointer Size

Architecture	Pointer Size
16-bit	2 bytes
32-bit	4 bytes
64-bit	8 bytes

The larger the address space, the larger the pointer size.

LPC2148 Example

The LPC2148 uses an **ARM7TDMI-S 32-bit processor**.

Therefore:

- int *p;
- char *p;
- float *p;
-

sizeof(pointer) = 4 bytes for all pointer types.

Example:

```c
#include <stdio.h>

int main()
{
   int *p;

   printf("%d", sizeof(p));

   return 0;
}
```

Output: 4

Key Points

1. A pointer stores the address of a variable.
2. Pointer size depends on the system architecture, not the data type.
3. All pointer types have the same size on a given system.
4. On LPC2148 (32-bit ARM7), every pointer occupies **4 bytes**.
5. sizeof(pointer) gives the size required to store an address.

Why Pointers are Important in Embedded Systems

Pointers are used in embedded systems for:

- Direct hardware register access
- Memory mapped I/O
- Peripheral interfacing
- DMA operations
- Efficient memory usage
- Interrupt vector table handling
- Communication buffers

Example:

#define GPIO_DIR (*(volatile unsigned int *)0xE0028008)

Here pointer is used to access hardware register directly.

Definition of Pointer

A pointer is a variable that stores the memory address of another variable.

Syntax of Pointer

data_type *pointer_name;

Example of Pointer

```
#include<stdio.h>

int main()
{
   int x = 10;

   int *ptr;

   ptr = &x;

   printf("Value of x = %d\n", x);
   printf("Address of x = %p\n", &x);
   printf("Value stored in ptr = %p\n", ptr);
   printf("Value pointed by ptr = %d\n", *ptr);

   return 0;
```

```
}
```

Pointer Arithmetic

What is Pointer Arithmetic?

Pointer arithmetic means performing mathematical operations (like addition and subtraction) on pointers. Since a pointer stores a memory address, arithmetic on it moves that address forward or backward through memory.

The Golden Rule:

When you add n to a pointer, it does **not** add n bytes. It adds:

n × sizeof(data type)

So if ptr is an int* and int is 4 bytes:

- ptr + 1 moves **4 bytes** forward
- ptr + 2 moves **8 bytes** forward
- ptr + 3 moves **12 bytes** forward

Example Setup

```
int arr[] = {10, 20, 30, 40};
int *ptr = arr;   // ptr points to arr[0], address 1000
```

Expression	Address	Value
ptr + 0	1000	10
ptr + 1	1004	20
ptr + 2	1008	30
ptr + 3	1012	40

Operations Allowed on Pointers

1. Addition (ptr + n)
Moves the pointer forward by n elements. Does not change ptr itself.

```
int *p = ptr + 2;   // p points to address 1008, value = 30
```

2. Increment (ptr++)
Moves ptr forward by 1 element. This changes ptr permanently.

```
ptr++;   // ptr now points to 1004
```

3. Subtraction (ptr - n)
Moves the pointer backward by n elements.

```
ptr - 1;   // goes back one element (4 bytes)
```

4. Pointer Difference (ptr2 - ptr1)
Gives the number of elements between two pointers (not bytes).

```
ptr2 - ptr1 = 2   // means 2 elements apart
```

5. Dereference (ptr)
Reads the value stored at the pointer's current address.

```
*ptr = 10   // reads the value at whatever address ptr holds
```

Array Indexing = Pointer Arithmetic

These two are **identical** in C:

```
arr[2]      // array notation
*(arr + 2)   // pointer arithmetic notation
```

Both go to address 1008 and return value 30.

What is NOT Allowed

Operation	Reason
ptr + ptr	Adding two addresses makes no sense
ptr × n	Multiplication on addresses is meaningless
ptr / n	Division on addresses is meaningless

Size Depends on Data Type

Type	Size	ptr+1 moves by
char*	1 byte	1 byte
int*	4 bytes	4 bytes
float*	4 bytes	4 bytes
double*	8 bytes	8 bytes

Types of Pointers

1. Null Pointer
2. Void Pointer
3. Wild Pointer
4. Dangling Pointer
5. Function Pointer
6. Pointer to Pointer
7. Constant Pointer
8. Pointer to Constant
9. Constant Pointer to Constant
10. Array Pointer
11. Structure Pointer
12. File Pointer
13. Near Pointer
14. Far Pointer
15. Huge Pointer

1)Null Pointer

A **null pointer** is a pointer that **does not point to any valid memory location**. It is assigned the special value **NULL**.

int *ptr = NULL;

Here, ptr is not pointing to any variable or memory location.

Why Do We Use a Null Pointer?

A null pointer is used to indicate:

- The pointer is not initialized yet.
- The pointer currently points to nothing.
- A memory allocation has failed.
- The end of a linked data structure has been reached.

Syntax

data_type *pointer_name = NULL;

Example:

int *ptr = NULL;
char *p = NULL;
float *fp = NULL;

Example 1

```c
#include <stdio.h>

int main()
{
   int *ptr = NULL;

   printf("%p", ptr);

   return 0;
}
```

Output

0x0

or

(nil)

depending on the compiler.

Memory Representation

int *ptr = NULL;

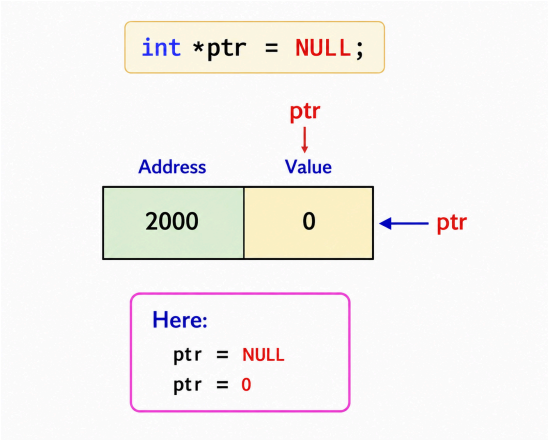
Address	Value

2000	0

Here:

ptr = NULL
ptr = 0

The pointer contains address **0**, which means it does not point to a valid object.



Example 2: Assigning a Valid Address Later

```c
#include <stdio.h>

int main()
{
  int num = 25;

  int *ptr = NULL;

  ptr = &num;

  printf("%d", *ptr);

  return 0;
}
```

Output : 25

Explanation

Initially: ptr = NULL

After: ptr = #

ptr stores the address of num.

Null Pointer vs Wild Pointer

Null Pointer

int *ptr = NULL;

- Explicitly initialized.
- Points to no valid memory.
- Safer because we know it is invalid.

Wild Pointer

int *ptr;

- Not initialized.
- Contains a garbage address.
- Very dangerous.

Example:

ptr = 458796

(garbage value)

Dereferencing a Null Pointer

int *ptr = NULL;

printf("%d", *ptr);

Result:

Runtime Error / Program Crash because there is no valid data at address 0.

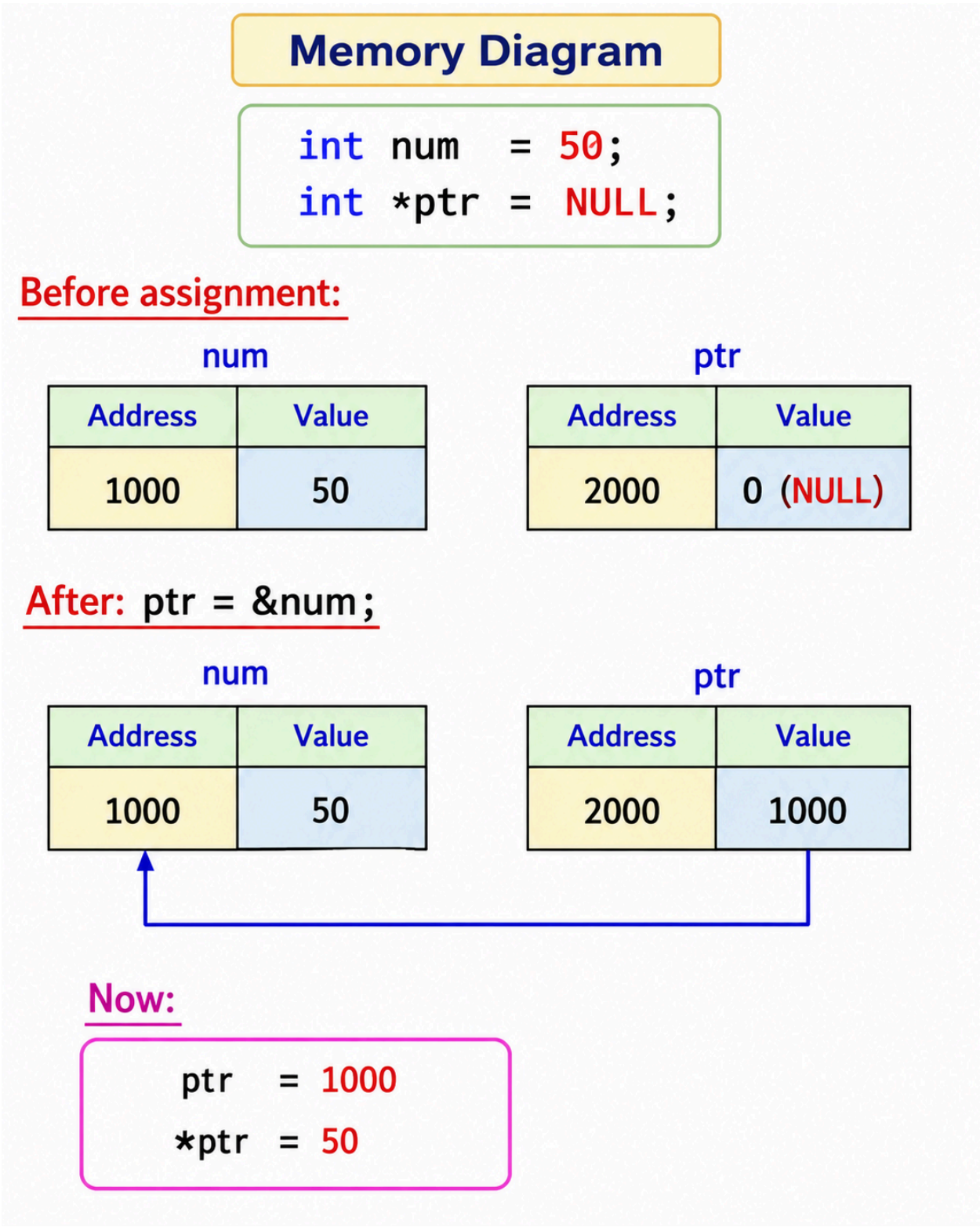
Checking for NULL Before Dereferencing

```
if(ptr != NULL)
{
```

```
    printf("%d", *ptr);
}
else
{
    printf("Pointer is NULL");
}
```

Output
Pointer is NULL. This is the safe way.

Memory Diagram



Practical Example: Dynamic Memory Allocation

```c
int *ptr;

ptr = (int *)malloc(sizeof(int));

if(ptr == NULL)
{
   printf("Memory allocation failed");
}
```

If memory allocation fails, malloc() returns NULL.

Important Points

1. A null pointer points to no valid memory location.
2. It is represented by NULL.
3. NULL is typically equivalent to address 0.
4. Dereferencing a null pointer causes a runtime error.
5. Always check ptr != NULL before dereferencing.
6. Null pointers are safer than uninitialized (wild) pointers.

2)Wild Pointer

A **wild pointer** is a pointer that has been declared but **not initialized**. Since it contains a garbage address, using it can lead to unpredictable behavior, crashes, or program errors.

Definition

> A wild pointer is an uninitialized pointer that points to an unknown (garbage) memory location.

Example of Wild Pointer

```c
#include<stdio.h>

int main()
{
   int *ptr;   // Wild Pointer

   printf("%u", ptr);

   return 0;
```

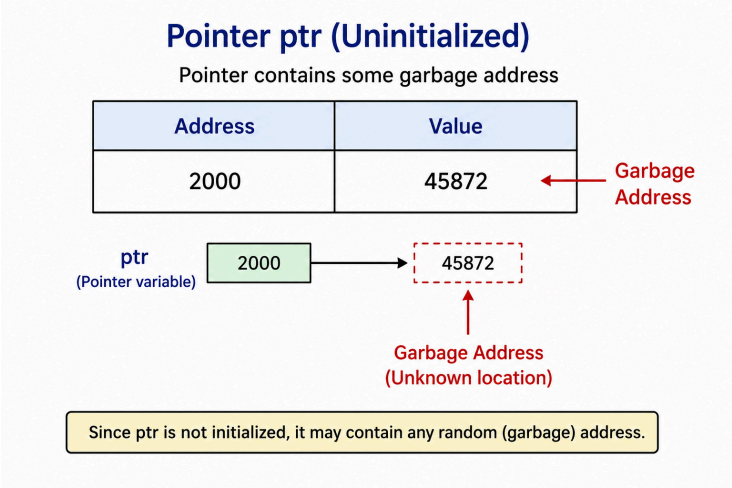
```
}
```

Explanation

int *ptr;

- Memory is allocated for ptr.
- No valid address is assigned.
- ptr contains a garbage value.

Example:



Address Value

2000 45872 ← Garbage Address

Here:

ptr = 45872; // Unknown address

This is called a **wild pointer**.

Dangerous Use of Wild Pointer

```
#include<stdio.h>

int main()
{
  int *ptr;     // Wild Pointer
```

```
    *ptr = 100;    // Dangerous

    return 0;
}
```

What Happens?

*ptr = 100;

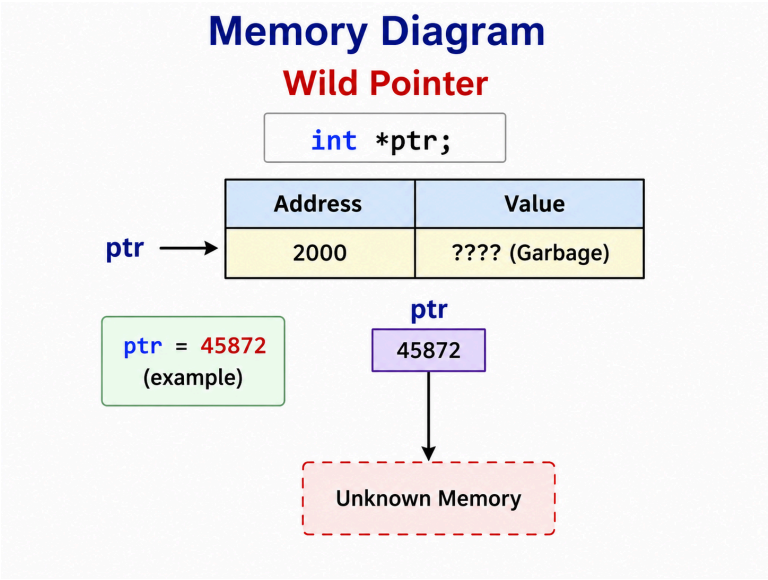
The program tries to store 100 at a random memory location.

Possible results:

- Program crash
- Segmentation fault
- Data corruption
- Unpredictable output

Memory Diagram

Wild Pointer



Since the address is unknown, dereferencing it is unsafe.

How to Avoid Wild Pointers

Method 1: Initialize to NULL

int *ptr = NULL;

A NULL pointer points nowhere and is safer than a wild pointer.

Method 2: Assign Valid Address

int num = 10;

int *ptr = #

This is a valid pointer.

Wild Pointer vs NULL Pointer

Wild Pointer	NULL Pointer
Contains garbage address	Contains address 0
Dangerous	Safe
Unpredictable behavior	Can be checked before use
May crash program	Does not point anywhere

Wild Pointer

int *ptr;

NULL Pointer

int *ptr = NULL;

Wild Pointer vs Dangling Pointer

Wild Pointer

Pointer never initialized.

int *ptr;

Dangling Pointer

Pointer points to memory that has already been freed.

```
int *ptr = (int *)malloc(sizeof(int));

free(ptr);   // ptr becomes dangling
```

Example Showing Safe Practice

```c
#include<stdio.h>

int main()
{
   int *ptr = NULL;

   if(ptr == NULL)
   {
      printf("Pointer is NULL");
   }

   return 0;
}
```

Output

Pointer is NULL

3)Dangling Pointer

A **dangling pointer** is a pointer that points to a memory location that is no longer valid.

Definition

A dangling pointer is a pointer that still contains the address of a memory location after that memory has been deallocated, deleted, or gone out of scope.

How Does a Dangling Pointer Occur?

A dangling pointer is commonly created in three situations:

1. After free()
2. After a local variable goes out of scope
3. After deleting dynamic memory (free() in C)

Case 1: Dangling Pointer After free()

Example

```c
#include<stdio.h>
#include<stdlib.h>

int main()
{
   int *ptr;

   ptr = (int *)malloc(sizeof(int));

   *ptr = 100;

   free(ptr);

   printf("%d", *ptr);   // Dangerous

   return 0;
}
```

Explanation

Before free():

Heap Memory

Address	Value
5000	100

But ptr

Address	Value
2000	5000

After:

free(ptr);

The memory at address **5000** is returned to the system.

Heap Memory

Address	Value
5000	Released

But ptr

Address	Value
2000	5000

Pointer still contains **5000**.

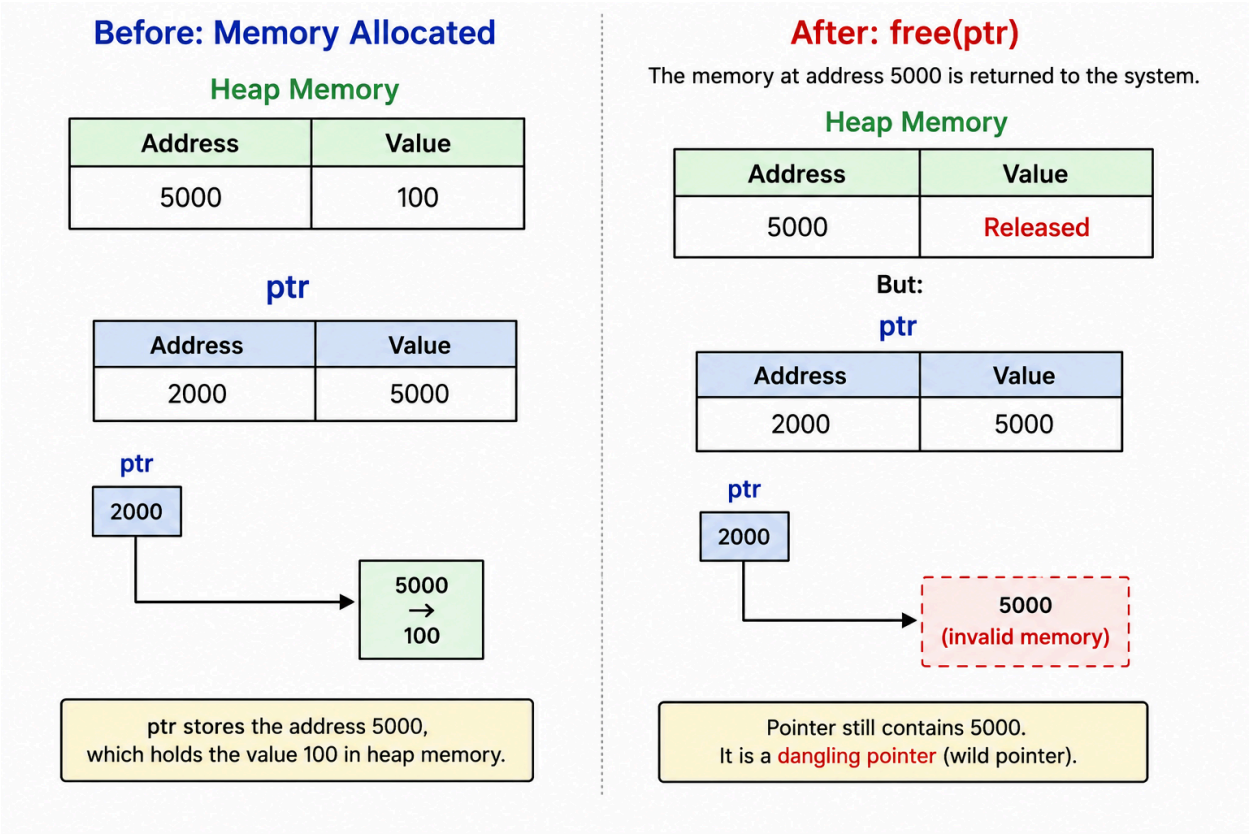
ptr —> 5000 (invalid memory)

Now ptr points to memory that no longer belongs to the program.

Therefore:

*ptr is dangerous. This is called a **dangling pointer**.

Memory Diagram



Pointer still stores 5000. Therefore it is a **dangling pointer**.

Correct Way

After freeing memory:

```
free(ptr);
ptr = NULL;
```

Now ptr,

Address	Value
2000	0

Pointer becomes NULL instead of dangling.

Case 2: Local Variable Goes Out of Scope

Example

```c
#include <stdio.h>

// Function returns an integer pointer
int* fun()
{
    // Local variable stored in the function's stack frame
    int x = 10;

    // Returning the address of the local variable
    // This is unsafe because 'x' is destroyed when the function returns
    return &x;
}

int main()
{
    // Declare an integer pointer
    int *ptr;

    // Store the address returned by the function
    ptr = fun();

    // Dereferencing ptr is unsafe because the memory of 'x'
    // is no longer valid after fun() returns
    printf("%d", *ptr);

    return 0;
}
```

Explanation

Inside function:

Address	Value
1000	10

When function returns:

return &x;

Variable x is destroyed.

Address 1000 becomes invalid.

But:

ptr = 1000

still remains.

ptr —-> Destroyed Memory

Hence ptr becomes a dangling pointer.

Wild Pointer vs Dangling Pointer

Wild Pointer	Dangling Pointer
Never initialized	Was valid earlier
Contains garbage address	Contains old invalid address
No valid memory ever assigned	Memory was assigned and later removed
Example: int *ptr;	Example: free(ptr);

Wild Pointer

int *ptr;
ptr = Store Garbage Value

Unknown address.

Dangling Pointer

int *ptr;

ptr = malloc(sizeof(int));

free(ptr);
ptr = old address

Invalid address.

NULL Pointer vs Dangling Pointer

NULL Pointer	Dangling Pointer
Points to address 0	Points to invalid address
Safe	Dangerous
Can be checked	May crash program

NULL Pointer
int *ptr = NULL;

Dangling Pointer
free(ptr);

Pointer still contains old address.

Example of Safe Programming

```c
#include<stdio.h>
#include<stdlib.h>

int main()
{
  int *ptr;

  ptr = (int *)malloc(sizeof(int));

  *ptr = 100;

  free(ptr);

  ptr = NULL;

  return 0;
}
```

After:

ptr = NULL;

the pointer is no longer dangling.

Summary Table

Pointer Type	Meaning
Valid Pointer	Points to valid memory
NULL Pointer	Points nowhere (address 0)
Wild Pointer	Uninitialized pointer
Dangling Pointer	Points to deallocated/invalid memory

4)Void Pointer

A **void pointer** is a special type of pointer that can store the address of **any data type**.

Definition:

> A void pointer is a generic pointer that can hold the address of any data type, but it cannot be dereferenced directly.

Syntax:
void *ptr;

Why Do We Need Void Pointers?

Normally, pointers are type-specific.

```
int num = 10;
int *iptr = &num;

float f = 12.5;
float *fptr = &f;
```

- int * can point to an integer.
- float * can point to a float.

Suppose we want **one pointer** that can point to different data types.

For this purpose, C provides: void *ptr;

Example 1: Pointing to Different Data Types

```c
#include <stdio.h>

int main()
{
    int num = 10;
    float f = 25.5;
    char ch = 'A';

    // Declare a void pointer
    // A void pointer can store the address of any data type
    void *ptr;

    // Store the address of the integer variable in the void pointer
    ptr = &num;

    // Typecast the void pointer to int pointer before dereferencing
    printf("%d\n", *(int *)ptr);

    // Store the address of the float variable in the void pointer
    ptr = &f;

    // Typecast the void pointer to float pointer before dereferencing
    printf("%.2f\n", *(float *)ptr);

    // Store the address of the character variable in the void pointer
    ptr = &ch;

    // Typecast the void pointer to char pointer before dereferencing
    printf("%c\n", *(char *)ptr);

    return 0;
}
```

Output

10
25.50
A

Memory Diagram

int num = 10;

void *ptr;

ptr = #

Memory of num :

Address	Value
1000	10

Memory of ptr:

Address	Value
2000	1000

Ptr —> 1000 → 10

Since ptr is a void pointer, it does not know the type of data stored at address 1000.

Why Can't We Dereference a Void Pointer Directly?

Consider,

```
void *ptr;
int num = 10;

ptr = &num;

printf("%d", *ptr);   // Error
```

Error

Invalid use of void pointer

Reason:

The compiler knows: ptr = 1000

but it does not know:

- Is it an int?
- Is it a float?
- Is it a char?

Without knowing the data type, it cannot determine how many bytes to read.

Therefore *ptr is illegal.

Type Casting is Required

Before dereferencing a void pointer, convert it to the appropriate type.

Integer Example

printf("%d", *(int *)ptr);

Explanation:

(int *)ptr

Convert void pointer to integer pointer.

Then: *(int *)ptr

Fetch integer value.

Step-by-Step Understanding

int num = 50;
void *ptr = #

num:

Address	Value
1000	50

ptr:

Address	Value
2000	1000

Now:

(int *)ptr

means:

Treat ptr as int pointer

Then:

*(int *)ptr becomes value at address 1000

Output: 50

Example 2: Void Pointer with Float

```
#include<stdio.h>

int main()
{
   float salary = 45000.75;

   void *ptr = &salary;

   printf("%.2f", *(float *)ptr);

   return 0;
}
```

Output

45000.75

Example 3: Void Pointer with Character

```
#include<stdio.h>

int main()
{
   char grade = 'A';

   void *ptr = &grade;

   printf("%c", *(char *)ptr);

   return 0;
}
```

Output

A

Generic Function Using Void Pointer

One major use of void pointers is creating generic functions.

Example

```c
#include<stdio.h>

void display(void *ptr, char type)
{
   if(type == 'i')
      printf("%d\n", *(int *)ptr);

   else if(type == 'f')
      printf("%.2f\n", *(float *)ptr);

   else if(type == 'c')
      printf("%c\n", *(char *)ptr);
}

int main()
{
   int a = 10;
   float b = 20.5;
   char c = 'X';

   display(&a,'i');
   display(&b,'f');
   display(&c,'c');

   return 0;
}
```

Output

```
10
20.50
X
```

This is why void pointers are called **generic pointers**.

Void Pointer Arithmetic

Not Allowed
void *ptr;

ptr++;

This is not standard C.

Reason:

The compiler does not know:

- How many bytes to move?
- 1 byte?
- 2 bytes?
- 4 bytes?

Therefore pointer arithmetic on void * is not defined by standard C.

Memory Allocation Functions Return Void Pointer

Functions like:

malloc()
calloc()
realloc()

return a void pointer.

Example:

int *ptr;

ptr = (int *)malloc(sizeof(int));

Why?

Because malloc() does not know what type of data you want.

It simply returns an address.

malloc()
↓
void *
↓
(int *) Conversion

Advantages of Void Pointer

1. Generic Programming
void *ptr;

Can store addresses of any data type.

2. Dynamic Memory Allocation

Used extensively with:

malloc()
calloc()
realloc()

3. Reusable Functions

Allows creation of generic functions.

5)Pointer to Pointer

Definition

A pointer storing address of another pointer is called pointer to pointer.

Syntax

int **ptr;

Example

```c
#include <stdio.h>

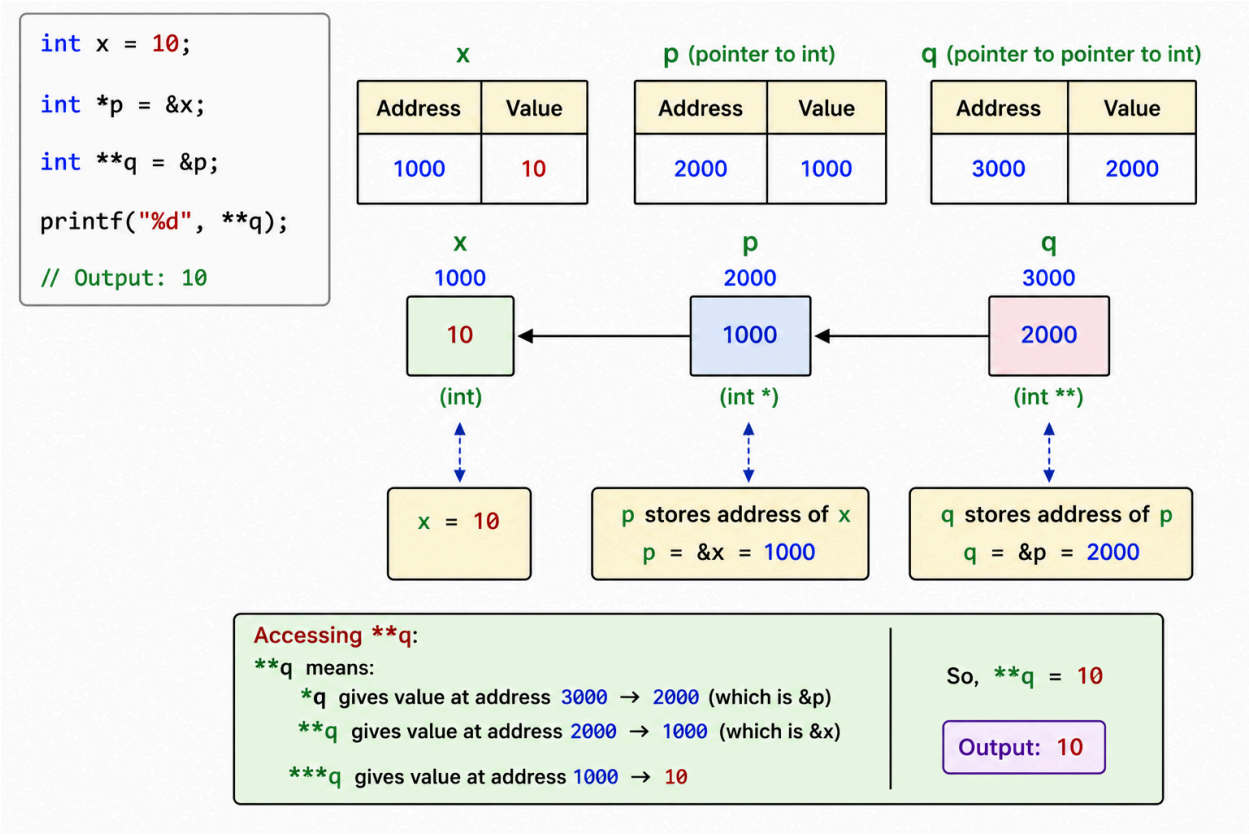
int main()
{
    // Declare an integer variable and initialize it with 10
    int x = 10;

    // Declare a pointer and store the address of x
    int *p = &x;

    // Declare a pointer-to-pointer and store the address of p
    int **q = &p;
```

```
    // Dereference q twice to access the value of x
    printf("%d", **q);

    return 0;
}
```



6)Array and Pointer Relationship

Arrays and pointers are closely related in C. In many expressions, the **array name behaves like a pointer to the first element of the array**.

Basic Example

int arr[5] = {10,20,30,40,50};

Memory Representation:

Address	Value

1000	10
1004	20
1008	30
1012	40
1016	50

Array name: arr

arr contains the address of the first element:

arr = &arr[0]

Therefore:

arr = 1000

Relationship Between Array and Pointer

int arr[5] = {10,20,30,40,50};

int *ptr = arr;

Equivalent to: int *ptr = &arr[0];

Memory: **arr**

Address	**Value**
1000	10
1004	20
1008	30
1012	40
1016	50

ptr:

Address	Value
2000	1000

Array Notation vs Pointer Notation

Consider: int arr[5] = {10,20,30,40,50};

Array Notation	Pointer Notation
arr[0]	*(arr+0)
arr[1]	*(arr+1)
arr[2]	*(arr+2)
arr[3]	*(arr+3)
arr[4]	*(arr+4)

Example: arr[2]

Means : *(arr+2)

Output: 30

Program 1: Access Array Using Pointer

```c
#include <stdio.h>

int main()
{
    // Declare and initialize an integer array
    int arr[5] = {10, 20, 30, 40, 50};

    // Declare an integer pointer
    int *ptr;

    // Store the address of the first element of the array in ptr
    ptr = arr;
```

```
    // Access and print the first element using the pointer
    printf("First Element = %d\n", *ptr);

    // Move the pointer to the second element and print its value
    printf("Second Element = %d\n", *(ptr + 1));

    // Move the pointer to the third element and print its value
    printf("Third Element = %d\n", *(ptr + 2));

    return 0;
}
```

Output

First Element = 10
Second Element = 20
Third Element = 30

How Pointer Arithmetic Works

Suppose: int arr[5] = {10,20,30,40,50};

Base address: 1000

Since integer occupies 4 bytes:

arr+0 = 1000
arr+1 = 1004
arr+2 = 1008
arr+3 = 1012
arr+4 = 1016

Thus: *(arr+3)

gives:40

Program 2: Print Array Using Pointer

```
#include<stdio.h>

int main()
{
   int arr[5] = {10,20,30,40,50};
```

```
    int *ptr;
    int i;

    ptr = arr;

    for(i=0;i<5;i++)
    {
        printf("%d ", *(ptr+i));
    }

    return 0;
}
```

Output
10 20 30 40 50

Program 3: Using Array Name as Pointer

```
#include<stdio.h>

int main()
{
    int arr[5] = {10,20,30,40,50};

    printf("%d\n", *arr);
    printf("%d\n", *(arr+1));
    printf("%d\n", *(arr+2));

    return 0;
}
```

Output
10
20
30

Array Name is a Constant Pointer
int arr[5];

You can do:

int *ptr = arr;

ptr++; → ✔ Valid

But:

arr++; → ❌ Invalid

Reason:

- ptr is a variable pointer.
- arr is a constant address (base address of array).

Difference Between Array and Pointer

Array	Pointer
Fixed memory block	Variable storing address
Size is entire array	Size is pointer size
Cannot be incremented	Can be incremented
Allocated at declaration	Can point anywhere

Example:

```
int arr[5];
int *ptr;
sizeof(arr)
```

Output: 20 (5 × 4 bytes)

sizeof(ptr)

Output (32-bit): 4

Output (64-bit): 8

Program 4: Sum Array Elements Using Pointer

```
#include<stdio.h>

int main()
{
```

```
    int arr[5] = {10,20,30,40,50};

    int *ptr;
    int i,sum=0;

    ptr = arr;

    for(i=0;i<5;i++)
    {
        sum += *(ptr+i);
    }

    printf("Sum = %d",sum);

    return 0;
}
```

Output

Sum = 150

Passing Array to Function Using Pointer

```
#include<stdio.h>

void display(int *ptr, int n);

int main()
{
    int arr[5] = {10,20,30,40,50};

    display(arr,5);

    return 0;
}

void display(int *ptr, int n)
{
    int i;

    for(i=0;i<n;i++)
    {
        printf("%d ", *(ptr+i));
    }
```

```
}
```

Output
10 20 30 40 50

Memory Diagram
int arr[5]={10,20,30,40,50};

Array Notation	Value	Address	Pointer Notation
arr[0]	10	1000	*(arr+0)
arr[1]	20	1004	*(arr+1)
arr[2]	30	1008	*(arr+2)
arr[3]	40	1012	*(arr+3)
arr[4]	50	1016	*(arr+4)

*(arr+2)

arr+2 → 1008

Value at 1008 = 30

Therefore: *(arr+2) = 30

Different Ways to Access Array

1. arr[i]
2. *(arr+i)
3. *(ptr+i)
4. ptr[i]
5. i[arr]
6. i[ptr]

Why i[arr] Works?

Because internally arr[i] is converted into *(arr+i).

Similarly: i[arr] becomes *(i+arr).

Since addition is commutative: arr+i == i+arr

Therefore: arr[i] == i[arr]

Pointer Subtraction Example

```
#include<stdio.h>

int main()
{
    int arr[5];

    int *p1 = &arr[1];
    int *p2 = &arr[4];

    printf("%d", p2-p1);

    return 0;
}
```

Output: 3

Passing Address to Function (Call by Reference)

Definition

In call by reference, address of variables is passed to function. Function modifies original variables.

Example 1: Swapping Numbers

```
#include<stdio.h>

void swap(int *a, int *b)
{
    int temp;

    temp = *a;
    *a = *b;
    *b = temp;
}
```

```c
int main()
{
    int x = 10;
    int y = 20;

    printf("Before Swap: %d %d
", x,y);

    swap(&x,&y);

    printf("After Swap: %d %d", x,y);

    return 0;
}
```

Example 2: Updating Variable Using Pointer

```c
#include<stdio.h>

void update(int *ptr)
{
    *ptr = 100;
}

int main()
{
    int x = 10;

    update(&x);

    printf("%d", x);

    return 0;
}
```

Example 3: Passing Array to Function Using Pointer

```c
#include<stdio.h>

void display(int *ptr, int size)
```

```
{
    int i;

    for(i=0;i<size;i++)
    {
        printf("%d ", *(ptr+i));
    }
}

int main()
{
    int arr[5] = {1,2,3,4,5};

    display(arr,5);

    return 0;
}
```

Example 4: Reverse Array Using Pointer

```
#include<stdio.h>

void reverse(int *ptr, int size)
{
    int i;

    for(i=size-1;i>=0;i--)
    {
        printf("%d ", *(ptr+i));
    }
}

int main()
{
    int arr[5] = {10,20,30,40,50};

    reverse(arr,5);

    return 0;
}
```

Pointer and String

```c
#include<stdio.h>

int main()
{
   char *str = "Embedded";

   printf("%s", str);

   return 0;
}
```

7)Array Pointer

Definition:

A pointer pointing to entire array is called array pointer.

Syntax

int (*ptr)[5];

Example

```c
#include <stdio.h>

int main()
{
    // Declare and initialize an integer array
    int arr[5] = {1, 2, 3, 4, 5};

    // Declare a pointer to an array of 5 integers
    // Store the address of the entire array in ptr
    int (*ptr)[5] = &arr;

    // Dereference ptr to access the array
    // [2] accesses the third element of the array
    printf("%d", (*ptr)[2]);

    return 0;
}
```

8)Array of Pointers

An **array of pointers** is an array in which **each element is a pointer**. Each pointer can store the address of a variable or another memory location.

Syntax

data_type *array_name[size];

Example

```c
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 20;
    int c = 30;

    // Declare an array of 3 integer pointers
    int *ptr[3];

    // Store the addresses of variables in the pointer array
    ptr[0] = &a;
    ptr[1] = &b;
    ptr[2] = &c;

    // Access the values using the pointers
    printf("%d\n", *ptr[0]);
    printf("%d\n", *ptr[1]);
    printf("%d\n", *ptr[2]);

    return 0;
}
```

9)Function Pointer

Definition

A pointer storing address of function is called function pointer.

Syntax

return_type (*pointer_name)(arguments);

Example

```c
#include <stdio.h>

// Function to display a message
void display()
{
    printf("Hello");
}

int main()
{
    // Declare a function pointer that points to a function
    // returning void and taking no arguments
    void (*ptr)();

    // Store the address of the display function in ptr
    ptr = display;

    // Call the display function using the function pointer
    ptr();

    return 0;
}
```

Function Pointer in Embedded Systems

Used in:

- Interrupt Vector Table
- Callback Functions
- RTOS
- Driver Abstraction
- State Machines

10)Constant Pointer

Definition

Pointer value cannot change.

Syntax

int *const ptr = &x;

Example

```c
#include <stdio.h>

int main()
{
    int x = 10;
    int y = 20;

    // Declare a constant pointer that stores the address of x
    // The pointer cannot point to another address
    int *const ptr = &x;

    // Change the value of x using the pointer
    *ptr = 100;

    // ptr = &y;  // Error: constant pointer cannot be reassigned

    // Print the value pointed to by ptr
    printf("%d", *ptr);

    return 0;
}
```

11)Pointer to Constant

Definition
Value cannot be modified through pointer.

Syntax

const int *ptr;

Example

```c
#include <stdio.h>

int main()
{
    // Declare and initialize an integer variable
    int x = 10;
```

```
    // Declare a pointer to a constant integer
    // The value of x cannot be modified using ptr
    const int *ptr = &x;

    // *ptr = 20;  // Error: cannot modify the value through ptr

    // Access and print the value of x using ptr
    printf("%d", *ptr);

    return 0;
}
```

12)Constant Pointer to Constant

Definition

Neither address nor value can change.

Syntax : const int *const ptr = &x;

Difference Between Constant Pointer and Pointer to Constant

Feature	Constant Pointer	Pointer to Constant
Address change	Not allowed	Allowed
Value change	Allowed	Not allowed

13)Structure Pointer

```
#include<stdio.h>

struct Student
{
   int roll;
};

int main()
```

```
{
    struct Student s1 = {10};

    struct Student *ptr = &s1;

    printf("%d", ptr->roll);

    return 0;
}
```

14)Returning Pointer from Function

```
#include<stdio.h>

int* fun()
{
    static int x = 10;

    return &x;
}

int main()
{
    int *ptr;

    ptr = fun();

    printf("%d", *ptr);

    return 0;
}
```

Dangerous Pointer Return

```
int* fun()
{
    int x = 10;

    return &x;
}
```

Danger:

Returning address of local variable.

Memory Mapped I/O Using Pointer

Embedded Systems Example

```
#define GPIO (*(volatile unsigned int *)0xE0028000)

int main()
{
   GPIO = 0xFF;

   return 0;
}
```

volatile with Pointer

Definition : volatile tells compiler that value may change unexpectedly.

Example :volatile int *ptr;

Importance in Embedded Systems

Used for:

- Hardware registers
- ISR variables
- Shared memory
- Peripheral status registers

Pointer to Volatile

volatile int *ptr;
Value may change unexpectedly.

Volatile Pointer

int * volatile ptr;
Pointer itself may change unexpectedly.

Tricky Pointer Expressions

Example 1

```
int x = 10;
int *ptr = &x;

printf("%d", ++*ptr);
```

Output: 11

Example 2

```
int arr[3] = {10,20,30};
int *ptr = arr;

printf("%d", *ptr++);
```

Output: 10

Example 3

printf("%d", (*ptr)++);
Value increments after use.

Example 4

printf("%d", *++ptr);
Pointer increments first.

Precedence Table

Expression	Meaning
*ptr++	*(ptr++)
++*ptr	++(*ptr)

*++ptr	*(++ptr)
(*ptr)++	value increment

Near, Far and Huge Pointer

These are old DOS-based memory model pointers.

Pointer	Description
Near	16-bit offset
Far	Segment + Offset
Huge	Modified far pointer

Mostly obsolete now.

Practice Questions

Basic Questions

1. What is pointer?
2. Why pointers are used?
3. Difference between pointer and array?
4. What is dereferencing?
5. What is null pointer?
6. What is dangling pointer?
7. What is wild pointer?
8. What is void pointer?
9. What is function pointer?
10. Why volatile used with pointers?

Advanced Questions

1. Difference between constant pointer and pointer to constant?
2. Explain memory mapped I/O.
3. Explain pointer arithmetic.
4. Why pointer arithmetic depends on datatype?

5. Difference between malloc and calloc?
6. What is double pointer?
7. Explain callback function.
8. What happens when free() called twice?
9. Why function pointer used in interrupt table?
10. Explain volatile pointer.

Practice Programs

1. Swap two numbers using pointer.
2. Reverse array using pointer.
3. Print array using pointer arithmetic.
4. Find largest number using pointer.
5. Create dynamic array using malloc.
6. Pass pointer to function.
7. Create pointer to structure.
8. Implement function pointer.
9. Access 2D array using pointer.
10. Create embedded register access using pointer.

MCQs
Q1.

```
int x = 10;
int *ptr = &x;

printf("%d", *ptr++);
```

Output?

A) 10
B) 11
C) Address
D) Error

Q2.

```
int x = 10;
int *ptr = &x;

printf("%d", ++*ptr);
```

What will be output of given code?

A) 10
B) 11
C) Address
D) Error

Q3.

Which pointer stores garbage address?

A) Null pointer
B) Void pointer
C) Wild pointer
D) Dangling pointer

Q4.

Which pointer can store any datatype address?

A) int pointer
B) void pointer
C) null pointer
D) dangling pointer

Q5.

Size of char pointer and int pointer on 32-bit system?

A) Different
B) Same
C) Depends on compiler only
D) Undefined